

Lektion 26

SQL

Mål

Efter endt undervisning kan de studerende:

- Anvende tidligere lærte materiale i C#
- Kendskab til Repositories og models i C# og brugen heraf

Agenda

1. Recap
2. Portfolio 7
3. Models
4. Repository
5. EFCore
6. Opgave

Library API

Portfolio opgave 7

Til opgave 7 skal i:

1. Planlægge data
2. udvikle EER
3. Opfyld normaliseringsform
4. minimum oprette InMemoryDatabase med EFCore

Models

Dto's var data transfer fra controllers til services. Models er data transfer fra services til database.

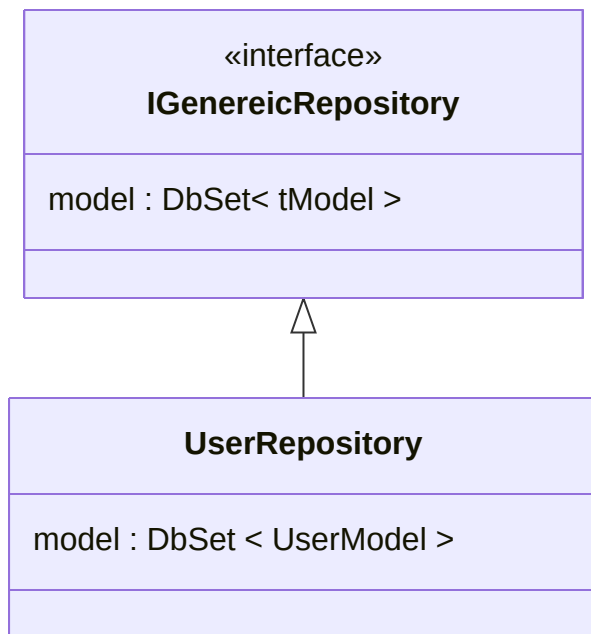
Models er tro mod databsen - hvor DTO er tro mod hvad brugeren skal se.

```
public class UserModel
{
    public string Name {set; get;} = string.Empty;
    public string Id {set; get;} = 0;
    public DateTime Created {get; set;}
}

public class UserDto
{
    public string Name {set; get;} = string.Empty;
    public string Id {set; get;} = 0;
}
```

Repository

Endnu et design pattern :D



```
public interface IGenericRepositories<tModel> where tModel : class
{
    void Add(tModel model);
    tModel? GetById(byte modelId);
    void Remove(byte modelId);
}

public class UserRepository : IGenericRepositories<UserModel>
{
    private readonly UserContext _userContext;
    public UserRepository(UserContext userContext){}
    public void Add(UserModel model){}
    public UserModel? GetById(byte modelId){ }
    public void Remove(byte modelId){}
}
```

Entity Framework Core

Faktisk et repository pattern mellem database og .net. Abstrahere database logik væk - så vi kan arbejde med database ved brug .net objekter.

2 modes, testing (InMemory) og production (database)

Tilføj package og installere tilføjelser:

1. dotnet new webapi
2. dotnet add package Microsoft.EntityFrameworkCore
3. dotnet tool install --global dotnet-ef
4. dotnet add package
Microsoft.EntityFrameworkCore.InMemory

For produktion:

1. dotnet add package provider
2. Lav context
3. dotnet ef migrations add InitialCreate
Opretter database første init.
4. dotnet ef database update
Updatere database - hvis nødvendigt.

Entity Framework Core

For DbSet operationer

```
public class UserContext : DbContext
{
    public UserContext
    (
        DbContextOptions<UserContext>
        options
    ) : base(options)
    {
    }

    public DbSet<UserModel> Users { get; set; }
}

static void Main()
{
    ...

    builder.Services.AddDbContext<UserContext>
    (
        options =>
        options.UseInMemoryDatabase("users.db"));
}
```

```
public UserRepository(UserContext userContext)
{
    _userContext = userContext;
}

public void Add(UserModel model)
{
    _userContext.Users.Add(model);
    _userContext.SaveChanges();
}

public UserModel? GetById(byte modelId)
{
    UserModel? user = _userContext.Users.
        FirstOrDefault(u => u.Id == modelId);

    if (user == null)
    {
        Console.WriteLine("User not found");
    }
}
```

Opgave 1 - Lav komplet CRUD

- Lav endpoint: `/api/users/`
 - GET
 - POST
- Lav endpoint: `/api/users/{userId}`
 - GET
 - PUT
 - DELETE

Opagve 2 - Udvid med Book entity

- Lav endpoint: `/api/books/`
 - POST
- Udvid med meningsfulde http verb.

Opgave 3 - Udvid med Bookshelves entity

- Udvid med meningsfulde http verb.

Opgave 4 - Udskift med MySQL

- Udskift provider med MySQL i stedet for InMemory